

CayleySpec: Formalizing Cayley Graph Spectral Theory and Hecke Operators in Lean 4

Tobias Weiss

tobias@weiss.com

TU Berlin (independent research)

July 14, 2026

Abstract

We present **CayleySpec**, a formalization in Lean 4 of the dictionary between Cayley graph spectra and Hecke eigenvalues of modular forms. The project formalizes three interconnected domains: (1) vertex-transitivity of Cayley graphs, (2) the spectral decomposition of Cayley adjacency operators via group characters and the Peter–Weyl theorem for finite groups, and (3) the definition and $\mathrm{SL}(2, \mathbb{Z})$ -invariance of Hecke operators acting on modular forms.

The formalization consists of approximately 1,000 lines of Lean code across five modules and builds successfully against mathlib4 v4.31.0 (2,958 jobs, 0 errors). All core theorems are fully proved, with no admitted theorems or sorries remaining.

The project serves dual purposes: as a self-contained case study in formalizing number-theoretic constructions in Lean, and as the formal-methods foundation for a broader research program connecting graph neural network expressivity to number-theoretic complexity via $\mathrm{SL}(2, \mathbb{F}_p)$ Cayley graphs.

1 Introduction

The connection between spectral graph theory and number theory runs deep. Perhaps the most striking example is the Ramanujan–Petersson conjecture, which bounds the eigenvalues of certain modular forms and translates, via the Eichler–Selberg trace formula, into bounds on the eigenvalues of regular graphs [?]. More concretely, the eigenvalues of Hecke operators—linear operators acting on spaces of modular forms—appear as the spectra of certain Cayley graphs of $\mathrm{GL}(2, \mathbb{Z}/p\mathbb{Z})$ [?].

This paper presents **CayleySpec**, a Lean 4 formalization that makes this connection precise at the level of formal proofs. The project formalizes:

- **Cayley graphs** and their vertex-transitivity (a known fact: every Cayley graph is vertex-transitive).
- **Spectral theory** of Cayley adjacency operators: for abelian groups, characters are eigenvectors; for general finite groups, the Peter–Weyl theorem decomposes the adjacency operator.
- **Hecke operators**: the operators T_n acting on modular forms of weight k , with a proof that T_n preserves the modular form property.

Motivation. This work sits within a larger research program connecting graph neural network (GNN) expressivity to number-theoretic complexity. Vertex-transitive Cayley graphs of $\mathrm{SL}(2, \mathbb{F}_p)$ are known to separate levels of the Weisfeiler–Leman (WL) hierarchy, implying that message-passing GNNs fail on certain graph isomorphism problems [?]. The long-term goal is to formalize the dictionary:

$$\text{Cayley graph spectra of } \mathrm{SL}(2, \mathbb{F}_p) \longleftrightarrow \text{Hecke eigenvalues of modular forms}$$

The present work completes the “Hecke side” of this dictionary: we formalize Hecke operators and prove their fundamental properties in Lean.

Contributions.

1. A formal proof that every Cayley graph is vertex-transitive, using the left-multiplication automorphism (42 lines).
2. A formal proof of the spectral decomposition of Cayley adjacency operators for finite groups: characters are eigenvectors for abelian groups, and the Peter–Weyl theorem gives the full decomposition (220 lines).
3. A formal proof that the Hecke operator T_n preserves the modular form property for $\mathrm{SL}(2, \mathbb{Z})$: slash-invariance under reduction, bijectivity of the reps indexing set, and holomorphy of the sum (513 lines).
4. A total of $\approx 1,000$ lines of Lean code, building cleanly against mathlib4 with 0 errors.

Outline. Section ?? provides background on Cayley graphs, spectral theory, and Hecke operators. Section ?? describes the formalization structure and key proof techniques. Section ?? evaluates the project: build statistics, proof status, and lessons learned. Section ?? discusses related work. Section ?? concludes and outlines future work.

2 Background

2.1 Cayley Graphs

Given a finite group G and a generating set $S \subseteq G$ (symmetric: $S = S^{-1}$, and $1 \notin S$), the *Cayley graph* $\mathrm{Cay}(G, S)$ has vertex set G and edges (g, gs) for all $g \in G$ and $s \in S$.

A key property of Cayley graphs is *vertex-transitivity*: for any two vertices $u, v \in G$, there exists a graph automorphism mapping u to v . The automorphism is simply left multiplication by $v \cdot u^{-1}$, which preserves adjacency because $(v \cdot u^{-1} \cdot g)^{-1} \cdot (v \cdot u^{-1} \cdot h) = g^{-1} \cdot h$.

2.2 Spectral Theory of Cayley Graphs

The *adjacency operator* A_S of a Cayley graph $\mathrm{Cay}(G, S)$ acts on functions $f : G \rightarrow \mathbb{C}$ by

$$(A_S f)(g) = \sum_{s \in S} f(g \cdot s).$$

For an *abelian* group G , the characters $\chi : G \rightarrow \mathbb{C}^\times$ form an eigenbasis:

$$A_S \chi = \left(\sum_{s \in S} \chi(s) \right) \chi.$$

For a general finite group G , the Fourier decomposition (Peter–Weyl theorem) [?] states that the regular representation decomposes as

$$\mathbb{C}[G] \cong \bigoplus_{\rho \in \text{Irr}(G)} \text{End}(\mathbb{C}^{\dim \rho}),$$

where the adjacency element $\sum_{s \in S} s \in \mathbb{C}[G]$ acts on each irreducible component ρ as $\sum_{s \in S} \rho(s)$.

2.3 Hecke Operators

For the modular group $\Gamma = \text{SL}(2, \mathbb{Z})$ acting on the upper half-plane $\mathbb{H}$, a *modular form* of weight k is a holomorphic function $f : \mathbb{H} \rightarrow \mathbb{C}$ satisfying

$$f\left(\frac{a\tau + b}{c\tau + d}\right) = (c\tau + d)^k f(\tau) \quad \text{for all } \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \Gamma,$$

and bounded at the cusp ∞ .

The n -th *Hecke operator* T_n acts on modular forms by

$$(T_n f)(\tau) = n^{k-1} \sum_{\substack{ad=n, a>0 \\ 0 \leq b < d}} d^{-k} f\left(\frac{a\tau + b}{d}\right).$$

An equivalent formulation uses the action of $\text{GL}(2, \mathbb{R})$ on functions $f : \mathbb{H} \rightarrow \mathbb{C}$ via the *slash operator*:

$$(f |_k g)(\tau) = \det(g)^{k-1} (c\tau + d)^{-k} f(g\tau) \quad \text{for } g = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

Then T_n can be written as a sum over representatives of the double coset $\Gamma \begin{pmatrix} n & 0 \\ 0 & 1 \end{pmatrix} \Gamma$:

$$T_n(f) = \sum_{M \in \text{reps}(n)} f |_k \Delta(M),$$

where $\text{reps}(n)$ is a set of representatives of the right cosets of Γ in $\Gamma \begin{pmatrix} n & 0 \\ 0 & 1 \end{pmatrix} \Gamma$, and $\Delta(M)$ embeds the integer matrix M into $\text{GL}(2, \mathbb{R})$.

3 Formalization

3.1 Project Structure

The formalization is organized into five modules:

File	Content	Lines
Defs.lean	IsVertexTransitive predicate	36
CayleyProp.lean	Vertex-transitivity proof	76
Spectrum.lean	Adjacency operator, character eigenvectors	140
Fourier.lean	Peter–Weyl theorem, regular representation	220
Hecke.lean	Hecke operators, modular form preservation	513

Table 1: Project module structure.

The project targets Lean 4.31.0 with mathlib4 v4.31.0, pinned via `lean-toolchain` and locked via `lake-manifest.json`.

3.2 Vertex-Transitivity of Cayley Graphs

The proof that every Cayley graph is vertex-transitive is a straightforward 42-line argument.

The key definition:

)

```
1 def IsVertexTransitive {V : Type*} (G : SimpleGraph V) : Prop :=
2   v w : V, ( : G g G), v = w
```

The construction of the left-multiplication automorphism:

)

```
1 def leftMul {G : Type*} [Group G] (s : Set G) (g : G) :
2   mulCayley s g mulCayley s :=
3   { toFun := fun x => g * x
4     invFun := fun x => g⁻¹ * x
5     left_inv := by intro x; simp
6     right_inv := by intro x; simp
7     map_rel_iff' := by
8       intro u v; simp [mulCayley_adj, mul_inv_rev] }
```

The main theorem follows directly:

)

```
1 theorem mulCayley_vertexTransitive {G : Type*} [Group G] (s : Set G) :
2   IsVertexTransitive (mulCayley s) := by
3   intro u v
4   refine mulCayley.leftMul s (v * u⁻¹), ?_
5   simp [mulCayley.leftMul]
```

3.3 Spectral Decomposition

The adjacency operator is defined as a linear map on the function space $G \rightarrow \mathbb{C}$:

)

```
1 noncomputable def adjacencyOperator (S : Finset G) :
2   (G → ℂ) → (G → ℂ) :=
3   { toFun := fun f g => ∑ s ∈ S, f (g * s)
4     map_add' := by intro f f; ext g;
5               simp [Pi.add_apply, Finset.sum_add_distrib]
6     map_smul' := by intro c f; ext g;
7               simp [Pi.smul_apply, smul_eq_mul, Finset.mul_sum] }
```

The character eigenvector theorem for abelian groups:

)

```

1 theorem character_eigenvector ( : G * ) :
2   (adjacencyOperator S) (fun g ( g : )) =
3   ( s S, ( s : )) (fun g ( g : )) := by
4   ext g
5   calc
6     (adjacencyOperator S) (fun g ( g : )) g
7     = s S, (( (g * s) : )) := rfl
8   _ = s S, (( g : ) * ( s : )) := by
9     simp [MonoidHom.map_mul ]
10  _ = ( s S, ( s : )) * ( g : ) := by
11    simp [Finset.mul_sum, mul_comm]
12  _ = (( s S, ( s : )) (fun g ( g : ))) g := rfl

```

3.4 Peter–Weyl Theorem for Finite Groups

The Fourier module formalizes the Peter–Weyl theorem for finite groups over $\mathbb{C}$. The key construction is the isomorphism $\text{Hom}_G(\mathbb{C}[G], V) \cong V$ for any representation V :

)

```

1 noncomputable def regularHomEquiv {G : Type 0} [Group G] [Fintype G]
2   [Invertible (Fintype.card G : )] {V : Type 0} [AddCommGroup V]
3   [Module V] [FiniteDimensional V]
4   ( : Representation G V) :
5   IntertwiningMap (leftRegular G) [] V := ...

```

This yields the multiplicity formula: each irreducible representation ρ appears $\dim(\rho)$ times in the regular representation.

3.5 Hecke Operators

The Hecke operator formalization is the most substantial component (513 lines). We follow the approach of Diamond–Shurman [?], using the slash action on functions $\mathbb{H} \rightarrow \mathbb{C}$.

3.5.1 The Reps Set

For a positive integer n , the set $\text{reps}(n)$ of fixed-determinant n matrices in normal form (upper-triangular, bounded entries) is finite. This is captured by the type `reps (n : ℤ)` with a `Fintype` instance from `mathlib`.

3.5.2 Slash-Invariance Under Reduction

The key lemma states that $\text{SL}(2, \mathbb{Z})$ -invariant functions are unaffected by the reduction algorithm on fixed-determinant matrices:

)

```

1 theorem slash_invariant_under_reduce {m : } (hm : m 0)
2   (hf : ( : SL(2, )), f [k] ( : GL (Fin 2) ) = f)
3   (A : FixedDetMatrix (Fin 2) m) :
4   f [k] (toGL hm A) = f [k] (toGL hm (FixedDetMatrices.reduce A)) := ...

```

The proof uses `exists_smul_reduce`: every fixed-determinant matrix is in the $\text{SL}(2, \mathbb{Z})$ -orbit of its reduction. This lemma is proved by induction on the reduction algorithm (three cases: base positive, base non-positive, and step).

3.5.3 Bijectivity of the Reindexing Map

To reindex the sum defining T_n , we prove that the map $M \mapsto \text{reduce}(\text{mulRight}(M, \gamma))$ is a bijection on $\text{reps}(n)$. Injectivity is the main difficulty: it requires constructing a group element $\delta \in \text{SL}(2, \mathbb{Z})$ such that $\delta \cdot M_2 = M_1$, then applying the injectivity lemma `reps_inj`. The construction goes through several layers of matrix algebra:

```

1 lemma _inj {m : } (hm : m < 0) ( : SL(2, )) (M M : reps m)
2   (h_eq : reduce (mulRight M.1 ) = reduce (mulRight M.1 )) :
3   M = M := ...

```

The proof chains three helper lemmas:

1. `h_smul_mulRight`: $\delta \cdot (\text{mulRight}(A, \gamma)) = \text{mulRight}(\delta \cdot A, \gamma)$
2. `h_mulRight_inj`: $\text{mulRight}(A, \gamma) = \text{mulRight}(B, \gamma)$ implies $A = B$ (via right-cancelling γ^{-1})
3. `reps_inj`: if $A, B \in \text{reps}(m)$ are in the same $\text{SL}(2, \mathbb{Z})$ -coset, then $A = B$ (via entrywise comparison using bounds)

3.5.4 $\text{SL}(2, \mathbb{Z})$ -Invariance of the Hecke Operator

The main invariance theorem:

```

1 theorem heckeOperator_slash ( : SL(2, )) (n : ) (hn : n < 0)
2   (hf : ( : SL(2, )), f [k] ( : GL (Fin 2) ) = f) :
3   (heckeOperator f k n hn) [k] ( : GL (Fin 2) ) =
4   heckeOperator f k n hn := ...

```

The proof strategy is:

1. Distribute the slash action across the sum using `SlashAction.sum_slash`.
2. Rewrite each summand using `h_step`: $(f|_k \Delta(M))|_k \gamma = f|_k \Delta(\text{reduce}(\text{mulRight}(M, \gamma)))$.
3. Reindex the sum via the bijection φ using `Function.Bijective.sum_comp`.

3.5.5 Holomorphy

The proof that $T_n(f)$ is holomorphic (`MDifferentiable`) when f is holomorphic uses two lemmas from `mathlib`:

1. `MDifferentiable.slash`: if f is `MDifferentiable`, then $f|_k g$ is `MDifferentiable` for any $g \in \text{GL}(2, \mathbb{R})$.
2. `MDifferentiable.sum`: a finite `Finset` sum of `MDifferentiable` functions is `MDifferentiable`.

The proof is just three lines:

)

```

1 have h_holo : MDiff F := by
2   dsimp [F, heckeOperator]
3   apply MDifferentiable.sum
4   intro M hM
5   exact MDifferentiable.slash f.holo' k (toGL hnz M.1)

```

3.5.6 Boundedness at Cusps

The boundedness-at-cusps condition is proved without q -expansion theory, using a geometric argument available in `mathlib`. The proof proceeds in two stages:

1. **Boundedness at ∞ .** For each summand $f \mid_k \Delta(M)$, the matrix $\Delta(M)$ is upper-triangular at position $(1, 0)$ because $M \in \text{reps}(n)$ has $M_{1,0} = 0$. The lemma `IsBoundedAtImInfty.slash` then implies each summand is bounded at ∞ . Finite sums preserve boundedness via `Finset.induction_on` and the `BoundedAtFilter.add` instance.
2. **Transport to any cusp.** For any cusp $c \in \text{SL}(2, \mathbb{Z}) \backslash \mathbb{P}^1(\mathbb{R})$, the lemma `isBoundedAt_iff_exists_SL2Z` provides $\gamma \in \text{SL}(2, \mathbb{Z})$ such that $\gamma \cdot \infty = c$. The $\text{SL}(2, \mathbb{Z})$ -invariance of T_n (proved above) gives $T_n(f) \mid_k \gamma = T_n(f)$, bounding $T_n(f)$ at c via the q -expansion-free criterion.

The only external dependency is the existence of the `IsBoundedAtImInfty.slash` lemma in `mathlib`, which is available as of v4.31.0. No q -expansion machinery is required.

4 Evaluation

4.1 Build Statistics

Metric	Value
Total build jobs	2,958
Errors	0
Admitted theorems	0
Lint warnings	4 (Fourier.lean section variables)
Total code (Lean)	$\approx 1,000$ lines
Build time (inc. <code>mathlib</code>)	≈ 80 s (cached)

Table 2: Build statistics for `CayleySpec`.

4.2 Proof Status

All core theorems are fully proved, with no admitted theorems:

- `mulCayley_vertexTransitive`: every Cayley graph is vertex-transitive.
- `character_eigenvector`: characters of abelian groups are eigenvectors of the adjacency operator.
- `regularHomEquiv`: $\text{Hom}_G(\mathbb{C}[G], V) \cong V$ for any representation V .
- `regular_multiplicity`: each irreducible representation appears $\dim(\rho)$ times in the regular representation.

- `slash_invariant_under_reduce`: $\text{SL}(2, \mathbb{Z})$ -invariant functions are invariant under the reduction of fixed-determinant matrices.
- `heckeOperator_slash`: the Hecke operator T_n preserves $\text{SL}(2, \mathbb{Z})$ -invariance.
- `heckeOperator_modularForm`: T_n maps modular forms to modular forms (holomorphy and boundedness proved).

All theorems are proved using only `mathlib4 v4.31.0`, with no q -expansion machinery required.

4.3 Lessons Learned

The notation gap. A recurring challenge was the difference between `Fintype.sum` and `Finset.sum`. The notation $\sum M : T, f M$ desugars to `Finset.sum Finset.univ`, but in Lean, `M : T, ...` is syntactic sugar—it is not a separate constant. This meant that lemmas expecting `i s, ...` would not directly match goals using `M : T, ...` without explicit rewriting via `Finset.sum_univ` or similar.

Congruence and sums. Earlier versions of the `heckeOperator_slash` proof attempted to use `congr` or `rw` to rewrite inside a sum binder. The working approach was to apply `congr 1`; `funext M`; `exact ...`, which works because after `congr 1`, the goal is an equality of functions at each M .

Matrix algebra. The `φ_inj` proof required careful handling of matrix equations with $\text{SL}(2, \mathbb{Z})$ elements. The key trick was to use `ext : 1` to descend to the matrix level, then use `Matrix.mul_assoc` and right-cancellation via γ^{-1} .

Induction on the reduction algorithm. The `exists_smul_reduce` lemma required induction on `reduce_rec` with three cases. Each case required explicit construction of the $\text{SL}(2, \mathbb{Z})$ element and a calc chain verifying the equality. The modular group generators T (translation) and S (inversion) were essential.

5 Related Work

Formalization of modular forms. Bosman and others formalized modular forms in Lean [?], establishing the framework for slash actions, $\text{SL}(2, \mathbb{Z})$ invariance, and the graded ring of modular forms. The present work builds directly on this foundation, extending it with the definition and invariance proof of Hecke operators.

Formalization of spectral graph theory. To our knowledge, this is the first formalization of Cayley graph spectral theory in a proof assistant. Closest related work includes formal proofs of Cheeger’s inequality [?] and spectral clustering in Coq.

AI-assisted formalization. A distinctive aspect of this project is that the entire codebase was developed through an AI-assisted workflow using OpenCode’s Sisyphus agent. The agent decomposed the specification into sub-problems, delegated exploration and implementation to specialized sub-agents, and verified each step via the build system. This is part of a growing trend in using LLMs for interactive theorem proving [?, ?, ?].

6 Conclusion and Future Work

We have presented CayleySpec, a formalization of Cayley graph spectral theory and Hecke operators in Lean 4. The project proves:

- vertex-transitivity of Cayley graphs (42 lines),
- the spectral decomposition of Cayley adjacency operators via group characters and the Peter–Weyl theorem (360 lines),
- $\mathrm{SL}(2, \mathbb{Z})$ -invariance and holomorphy of Hecke operators (513 lines).

The build is clean (0 errors, 0 admitted theorems) and the code serves as both a self-contained formalization and a foundation for future work.

Future work. Several directions are planned:

1. **q -expansion theory:** The q -expansion machinery for modular forms (`Mathlib.NumberTheory.ModularForm`) is now imported and available. While the Hecke operator boundedness proof does not require q -expansions (the geometric proof suffices), the imported machinery enables future Fourier-analytic proofs such as the q -expansion principle and formulas for Hecke eigenform coefficients.
2. **Spectra of $\mathrm{SL}(2, \mathbb{F}_p)$ Cayley graphs:** The formalization of the Peter–Weyl theorem for finite groups can be extended to compute the spectrum of $\mathrm{SL}(2, \mathbb{F}_p)$ Cayley graphs via the character table of $\mathrm{SL}(2, \mathbb{F}_p)$.
3. **Hecke–Cayley dictionary:** The formal bridge between Hecke eigenvalues and Cayley graph spectra requires linking the n -dimensional representations of $\mathrm{GL}(2, \mathbb{Z}/p\mathbb{Z})$ with Hecke operators on modular forms of weight $k = 2$.
4. **WL hierarchy formalization:** The longer-term goal is to formalize the Weisfeiler–Leman hierarchy for graph neural networks, using vertex-transitive $\mathrm{SL}(2, \mathbb{F}_p)$ Cayley graphs as separating examples.

Availability. The formalization is freely available at:

<https://github.com/tobias-weiss-ai-xr/CayleySpec>

under the Apache 2.0 license.

References

- [1] A. Lubotzky. *Discrete Groups, Expanding Graphs and Invariant Measures*. Birkhäuser, 1994.
- [2] G. Davidoff, P. Sarnak, and A. Valette. *Elementary Number Theory, Group Theory, and Ramanujan Graphs*. Cambridge University Press, 2003.
- [3] M. Grohe. The descriptive complexity of graph neural networks. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2021.
- [4] J.-P. Serre. *Linear Representations of Finite Groups*. Springer, 1977.

- [5] F. Diamond and J. Shurman. *A First Course in Modular Forms*. Springer, 2005.
- [6] C. Bosman and the mathlib community. Modular forms in mathlib. https://leanprover-community.github.io/mathlib4_docs/Mathlib/NumberTheory/ModularForms/Basic.html, 2024.
- [7] K. Nakano. Formal proof of Cheeger’s inequality in Lean. Preprint, 2024.
- [8] E. First and T. Ringer. GPT for theorem proving: A survey. Preprint, 2023.
- [9] A. Thakur et al. Large language models for interactive theorem proving: A survey. Preprint, 2024.
- [10] A. Q. Jiang et al. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. Preprint, 2024.